



Digital IC Design

Verilog and Combinational Logic Examples

Taihai Chen
陈泰海

2024.12.27

Today' s arrangement



1. Self introduction – who am I?
2. Combinational logic examples with Verilog
3. Live demo of simple logic gate/module simulations in Vivado from AMD Xilinx
4. Q & A

In between these...

Interaction is strongly recommended.

If you have questions? Please ask anytime.

Note:

1. Teaching materials of my part in this course are mostly referenced from and inspired by Prof. Yubo Li (李宇波教授). So, thank you Prof Li!
2. Some of RTL code in this course are referenced from and/or inspired by www.vlsiverify.com and 《从算法到电路: 数字芯片算法的电路实现》 (*From algorithm to circuits: algorithms and implementation for digital chips* in English).



- Dr Taihai Chen
 - Principal Investigator (Technology), ZJU-HIC
 - Adjunct Fellow, Institute of VLSI, ZJU
- Education
 - PhD, University of Southampton, UK, 2010-15
 - MSc, University of Southampton, UK, 2009-10
 - BEng, University of Electronic Science and Technology of China, China, 2005-09
- Interests
 - Wireless Baseband Signal Processing (5G, 6G, etc), Digital IC Design
 - Knowledge Transfer, Commercialisation

- Work Experience

- | | | |
|---------------------------------------|-----------------------------|-------------------|
| • Principal Investigator (Technology) | @ ZJU-HIC | 2024.4 – Present |
| • Senior FAE | @ AccelerComm Ltd | 2022.6 – 2024.2 |
| • Solution Architect | @ AccelerComm Ltd | 2021.4 – 2022.5 |
| • Regional Business Director | @ AccelerComm Ltd | 2019.1 – 2021.3 |
| • Chief Operating Officer | @ AccelerComm Ltd | 2016.12 – 2018.12 |
| • Integrated Circuit Design Engineer | @ AccelerComm Ltd | 2016.12 – 2018.12 |
| • Post-doctoral Researcher | @ University of Southampton | 2014.5 – 2016.11 |

- Enterprise History

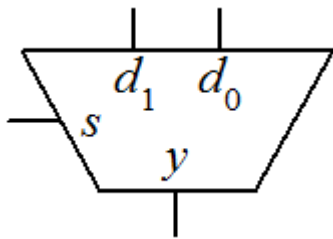
- Co-founder @ AccelerComm Ltd

Multiplexer (Mux)



- n -to-1 multiplexer
 - eg: 2-1 multiplexer

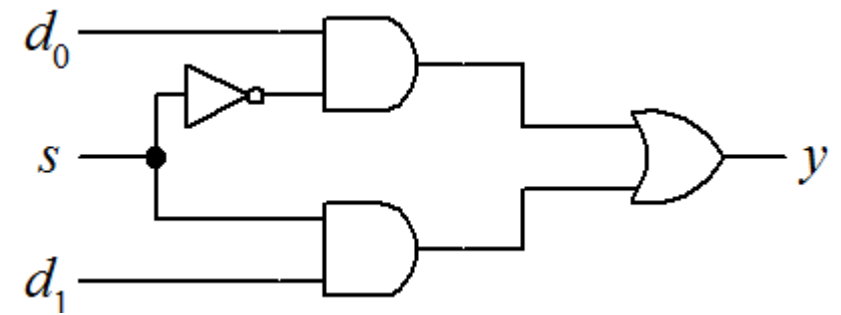
Symbol



Truth table

s	d ₁	d ₀	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Circuit

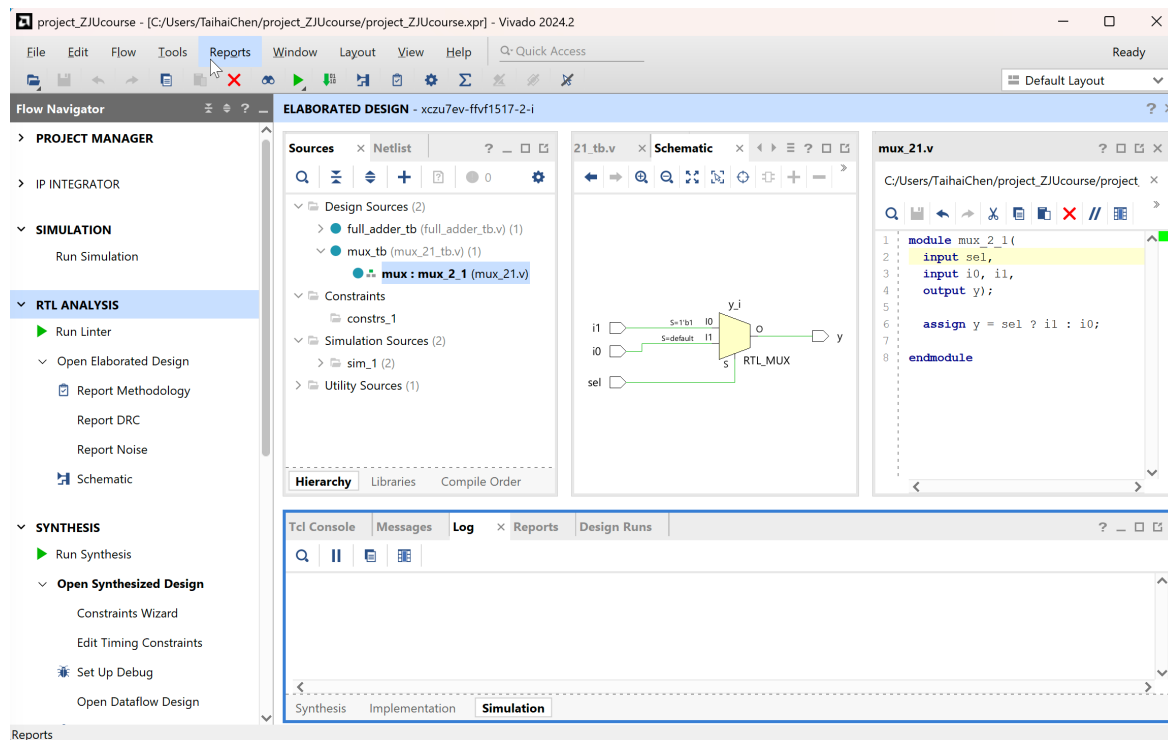


Two Styles in Xilinx Tools



- Schematic

- HDL



```
module mux_2_1(  
    input sel,  
    input i0, i1,  
    output y  
);
```

```
    assign y = sel ? i1 : i0;
```

```
endmodule
```

```
module mux_2_1(  
    input sel,  
    input i0, i1,  
    output y  
);
```

```
    wire sn, snd0, sd1;
```

```
    not U1 (sn, sel);  
    and U2 (snd0, i0, sn);  
    and U3 (sd1, i1, sel);  
    or    U4 (y, snd0, sd1);
```

```
endmodule
```

Two Styles in Xilinx Tools



project_ZJUcourse - [C:/Users/TaihaiChen/project_ZJUcourse/project_ZJUcourse.xpr] - Vivado 2024.2

File Edit Flow Tools Reports Window Layout View Run Help Quick Access

Synthesis Out-of-date details

Default Layout

Flow Navigator

- PROJECT MANAGER
 - Settings
 - Add Sources
 - Language Templates
 - IP Catalog
- IP INTEGRATOR
 - Create Block Design
 - Open Block Design
 - Generate Block Design
- SIMULATION
 - Run Simulation
- RTL ANALYSIS
 - Run Linter
 - Open Elaborated Design
- SYNTHESIS
 - Run Synthesis
 - Open Synthesized Design
- IMPLEMENTATION

SIMULATION - Behavioral Simulation - Functional - sim_1 - mux_tb

Scope Sources

Design Sources (2)

- mux_tb (mux_21_tb.v) (1)
 - mux: mux_2_1 (mux_21_1)
 - full_adder_tb (full_adder_tb.v)
- Constraints
- Simulation Sources (2)
- Utility Sources (1)

Hierarchy Libraries

mux_21_tb.v

```
1 module mux_tb;
2   reg i0, i1, sel;
3   wire y;
4
5   mux_2_1 mux(sel, i0, i1, y);
6   initial begin
7     $monitor("sel = %h: i0 = %h
8     i0 = 0; i1 = 1;
9     sel = 0;
10    #1;
11    sel = 1;
12    #1;
13  end
14 endmodule
```

Untitled 2

Name	Value
i0	0
i1	1
sel	1
y	1

Tcl Console

Messages Log

```
sel = 1: i0 = 0, i1 = 1 --> y = 1
xsim: Time (s): cpu = 00:00:06 ; elapsed = 00:00:08 . Memory (MB): peak = 1718.105 ; gain = 0.000
INFO: [USF-XSim-96] XSim completed. Design snapshot 'mux_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:07 ; elapsed = 00:00:17 . Memory (MB): peak = 1718.105 ; gain = 0.000
```

Type a Tcl command here

- Text file containing the mapping from a device independent HDL circuit net to the physical I/O pin. This allows Verilog (HDL) to be device independent.

```
net "ram0_data<35>" loc="ab25" | fast | iostandard=lvdci_33 | drive=12;
```

- Assigns bit 35 of the signal ram0_data to pin ab25
 - Specifies the i/o driver configured for fast slew rate with 3.3V LVTTL level
 - Specifies drive strength of 12mA
- Constraints may also include timing constraints.
- Don't worry - all constraints for the labkit have been defined
- For Vivado, xdc file are used (Xilinx Design Constraint)

A **specification** is an engineering contract that lists all the goals for a project:

- goals include area, power, throughput, latency, functionality, test coverage, costs (NREs and piece costs), ... Helps you figure out when you're done and how to make engineering tradeoffs. Later on, goals help remind everyone (especially management) what was agreed to at the outset!

- top-down design: partition the project into modules with well-defined interfaces so that each module can be worked on by a separate team. Gives the SW types a head start too!

(**Hardware/software codesign** is currently all the rage...)

- Example - a well defined Instruction Set Architecture (ISA) can last for generations ...

A behavioral model serves as an executable functional specification that documents the exact behavior of all the individual modules and their interfaces. Since one can run tests, this model can be refined and finally verified through simulation.

We need a way to talk about what hardware should do without actually designing the hardware itself, i.e., we need to separate behavior from implementation. We need a

Hardware Description Language

If we were then able to synthesize an implementation directly from the behavioral model, we'd be in good shape!

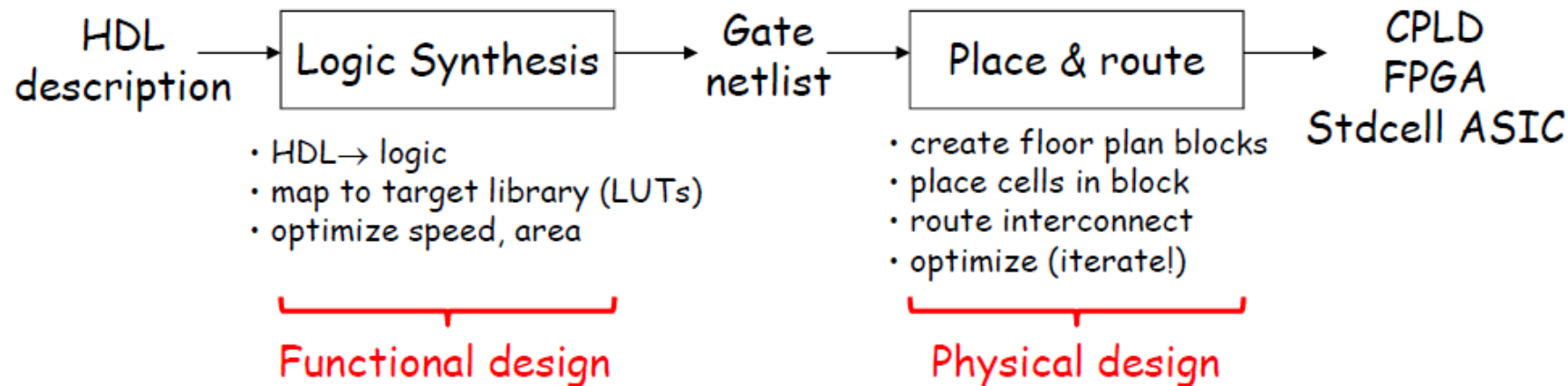
Using an HDL Description



So, we have an executable functional specification that

- documents exact behavior of all the modules and their interfaces
- can be tested & refined until it does what we want

An HDL description is the first step in a mostly automated process to build an implementation directly from the behavioral model



Since we're describing hardware, we'll need to represent the values that can appear on wires. Verilog uses a 4-valued logic:

Value	Meaning
0	Logic zero, "low"
1	Logic one, "high"
Z or ?	High impedance (tri-state buses)
X	Unknown value (simulation)

"X" is used by simulators when a wire hasn't been initialized to a known value or when the predicted value is an illegitimate logic value (e.g., due to contention on a tri-state bus).

Verilog also has the notion of "drive strength" but we can safely ignore this feature for our purposes.

Constant values can be specified with a specific width and radix:

```
123          // default: decimal radix, unspecified width
'd123        // 'd = decimal radix
'h7B         // 'h = hex radix
'o173        // 'o = octal radix
'b111_1011   // 'b = binary radix, "_" are ignored
'hxx         // can include X, Z or ? in non-decimal constants
16'd5        // 16-bit constant 'b0000_0000_0000_0101
11'h1X?      // 11-bit constant 'b001_XXXX_ZZZZ
```

By default constants are unsigned and will be extended with 0's on left if need be (if high-order bit is X or Z, the extended bits will be X or Z too). You can specify a signed constant as follows:

```
8'shFF       // 8-bit twos-complement representation of -1
```

To be absolutely clear in your intent **it's usually best to explicitly specify the width and radix.**

We have to provide **declarations*** for all our named wires (aka "nets"). We can create buses - indexed collections of wires - by specifying the allowable range of indices in the declaration:

```
wire a,b,z;           // three 1-bit wires
wire [31:0] memdata;  // a 32-bit bus
wire [7:0] b1,b2,b3,b4; // four 8-bit buses
wire [W-1:0] input;   // parameterized bus
```

Note that [0:7] and [7:0] are both legitimate but it pays to develop a convention and stick with it. Common usage is **[MSB:LSB]** where $MSB > LSB$; usually LSB is 0. Note that we can use an expression in our index declaration but the expression's value must be able to be determined at compile time. We can also build unnamed buses via concatenation:

```
{b1,b2,b3,b4} // 32-bit bus, b1 is [31:24], b2 is [23:16], ...
{4{b1[3:0]},16'h0000} // 32-bit bus, 4 copies of b1[3:0], 16 0's
```

* Actually by default undeclared identifiers refer to a 1-bit wire, but this means typos get you into trouble. Specify "``default_nettype none`" at the top of your source files to avoid this bogus behavior.

Boolean Operators



- **Bitwise operators** perform bit-oriented operations on vectors
 - $\sim(4'b0101) = \{\sim 0, \sim 1, \sim 0, \sim 1\} = 4'b1010$
 - $4'b0101 \& 4'b0011 = \{0\&0, 1\&0, 0\&1, 1\&1\} = 4'b0001$
- **Reduction operators** act on each bit of a single input vector
 - $\&(4'b0101) = 0 \& 1 \& 0 \& 1 = 1'b0$
- **Logical operators** return one-bit (true/false) results
 - $!(4'b0101) = 1'b0$

Bitwise

$\sim a$	NOT
$a \& b$	AND
$a b$	OR
$a \wedge b$	XOR
$a \sim \wedge b$ $a \wedge \sim b$	XNOR

Reduction

$\&a$	AND
$\sim \&a$	NAND
$ a$	OR
$\sim a$	NOR
$\wedge a$	XOR
$\sim \wedge a$ $\wedge \sim a$	XNOR

Logical

$!a$	NOT
$a \&\& b$	AND
$a b$	OR
$a == b$ $a != b$	[in]equality returns x when x or z in bits. Else returns 0 or 1
$a === b$ $a !== b$	case [in]equality returns 0 or 1 based on bit by bit comparison

*Note distinction between $\sim a$ and $!a$
when operating on multi-bit values*

Boolean Operators



- **^ is NOT exponentiation (**)**
- **Logical operator with z and x**
 - $4'bz0x1 === 4'bz0x1 = 1$ $4'bz0x1 === 4'bz001 = 0$
- **Bitwise operator with z and x**
 - $4'b0001 \& 4'b1001 = 0001$ $4'b1001 \& 4'bx001 = x001$

Bitwise	
$\sim a$	NOT
$a \& b$	AND
$a b$	OR
$a \wedge b$	XOR
$a \sim \wedge b$ $a \wedge \sim b$	XNOR

Reduction	
$\&a$	AND
$\sim \&a$	NAND
$ a$	OR
$\sim a$	NOR
$\wedge a$	XOR
$\sim \wedge a$ $\wedge \sim a$	XNOR

Logical	
$!a$	NOT
$a \&\& b$	AND
$a b$	OR
$a == b$ $a != b$	[in]equality returns x when x or z in bits. Else returns 0 or 1
$a === b$ $a !== b$	case [in]equality returns 0 or 1 based on bit by bit comparison

*Note distinction between $\sim a$ and $!a$
when operating on multi-bit values*

Other Operators



Conditional

$a ? b : c$	If a then b else c
-------------	--------------------

Relational

$a > b$	greater than
$a \geq b$	greater than or equal
$a < b$	Less than
$a \leq b$	Less than or equal

Arithmetic

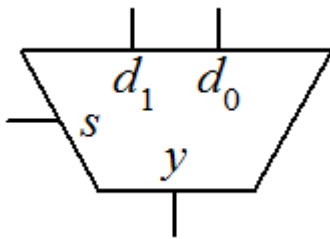
$-a$	negate
$a + b$	add
$a - b$	subtract
$a * b$	multiply
a / b	divide
$a \% b$	modulus
$a ** b$	exponentiate
$a \ll b$	logical left shift
$a \gg b$	logical right shift
$a \lll b$	arithmetic left shift
$a \ggg b$	arithmetic right shift

Multiplexer (Mux)



- n -to-1 multiplexer
 - eg: 2-1 multiplexer

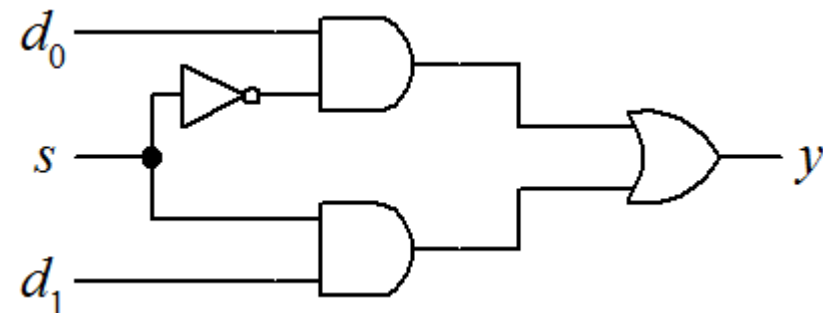
Symbol



Truth table

s	d ₁	d ₀	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Circuit



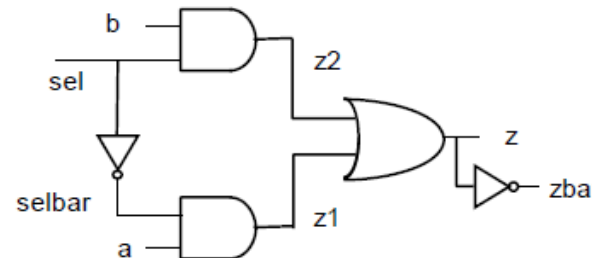
Basic Building Block:



In Verilog we design modules, one of which will be identified as our top-level module. Modules usually have named, directional ports (specified as `input`, `output` or `inout`) which are used to communicate with the module.

Don't forget this ";"

```
// 2-to-1 multiplexer with dual-polarity outputs
module mux2(input a,b,sel, output z,zbar);
    wire selbar,z1,z2;    // wires internal to the module
    // order doesn't matter - all statements are
    // executed concurrently!
    not i1(selbar,sel);    // inverter, name is "i1"
    and a1(z1,a,selbar);   // port order is (out,in1,in2,...)
    and a2(z2,b,sel);
    or o1(z,z1,z2);
    not i2(zbar,z);
endmodule
```



In this example the module's behavior is specified using Verilog's built-in Boolean modules: `not`, `buf`, `and`, `nand`, `or`, `nor`, `xor`, `xnor`. *Just say no!* We want to specify behavior, not implementation!

If we want to specify a behavior equivalent to combinational logic, use Verilog's operators and continuous assignment statements:

```
// 2-to-1 multiplexer with dual-polarity outputs
module mux2(input a,b,sel, output z,zbar);
    // again order doesn't matter (concurrent execution!)
    // syntax is "assign LHS = RHS" where LHS is a wire/bus
    // and RHS is an expression
    assign z = sel ? b : a;
    assign zbar = ~z;
endmodule
```

Conceptually `assign`'s are evaluated continuously, so whenever a value used in the RHS changes, the RHS is re-evaluated and the value of the wire/bus specified on the LHS is updated.

This type of execution model is called "**dataflow**" since evaluations are triggered by data values flowing through the network of wires and operators.

Hierarchy: Module Instances



Our descriptions are often hierarchical, where a module's behavior is specified by a circuit of module instances:

```
// 4-to-1 multiplexer
module mux4(input d0,d1,d2,d3, input [1:0] sel, output z);
    wire z1,z2;
    // instances must have unique names within current module.
    // connections are made using .portname(expression) syntax.
    // once again order doesn't matter...
    mux2 m1(.sel(sel[0]),.a(d0),.b(d1),.z(z1)); // not using zbar
    mux2 m2(.sel(sel[0]),.a(d2),.b(d3),.z(z2));
    mux2 m3(.sel(sel[1]),.a(z1),.b(z2),.z(z));
    // could also write "mux2 m3(z1,z2,sel[1],z,)" NOT A GOOD IDEA!
endmodule
```

Connections to module's ports are made using a syntax that specifies both the port name and the wire(s) that connects to it, so ordering of the ports doesn't have to be remembered ("**explicit**").

This type of hierarchical behavioral model is called "**structural**" since we're building up a structure of instances connected by wires. We often mix dataflow and structural modeling when describing a module's behavior.

Parameterized Modules



```
// 2-to-1 multiplexer, W-bit data
module mux2 #(parameter W=1) // data width, default 1 bit
    (input [W-1:0] a,b,
     input sel,
     output [W-1:0] z);
    assign z = sel ? b : a;
    assign zbar = ~z;
endmodule
```

```
// 4-to-1 multiplexer, W-bit data
module mux4 #(parameter W=1) // data width, default 1 bit
    (input [W-1:0] d0,d1,d2,d3,
     input [1:0] sel,
     output [W-1:0] z);
    wire [W-1:0] z1,z2;

    mux2 #(.W(W)) m1(.sel(sel[0]),.a(d0),.b(d1),.z(z1));
    mux2 #(.W(W)) m2(.sel(sel[0]),.a(d2),.b(d3),.z(z2));
    mux2 #(.W(W)) m3(.sel(sel[1]),.a(z1),.b(z2),.z(z));
endmodule
```

*could be an expression evaluable at compile time;
if parameter not specified, default value is used*

There are times when we'd like to use sequential semantics and more powerful control structures - these are available inside sequential `always` blocks:

```
// 4-to-1 multiplexer
module mux4(input a,b,c,d, input [1:0] sel, output reg z,zbar);
    always @(*) begin
        if (sel == 2'b00) z = a;
        else if (sel == 2'b01) z = b;
        else if (sel == 2'b10) z = c;
        else if (sel == 2'b11) z = d;
        else z = 1'bx;    // when sel is X or Z
        // statement order matters inside always blocks
        // so the following assignment happens *after* the
        // if statement has been evaluated
        zbar = ~z;
    end
endmodule

always @(*) blocks are evaluated whenever any value used inside
changes. Equivalently we could have written

always @(a, b, c, d, sel) begin ... end    // careful, prone to error!
```


We've been using `wire` declarations when naming nets (ports are declared as wires by default). However nets appearing on the LHS of assignment statements inside of `always` blocks *must* be declared as type `reg`.

I don't know why Verilog has this rule! I think it's because traditionally `always` blocks were used for sequential logic (the topic of next lecture) which led to the synthesis of hardware registers instead of simply wires. So this seemingly unnecessary rule really supports historical usage - the declaration would help the reader distinguish registered values from combinational values.

We can add the `reg` keyword to `output` or `inout` ports (we wouldn't be assigning values to `input` ports!), or we can declare nets using `reg` instead of `wire`.

```
output reg [15:0] result // 16-bit output bus assigned in always block
reg flipflop;           // declaration of 1-bit net of type reg
```


Chains of **if-then-else** statements aren't the best way to indicate the intent to provide an alternative action for every possible control value. Instead use **case**:

```
// 4-to-1 multiplexer
module mux4(input a,b,c,d, input [1:0] sel, output reg z,zbar);
  always @(*) begin
    case (sel)
      2'b00: z = a;
      2'b01: z = b;
      2'b10: z = c;
      2'b11: z = d;
      default: z = 1'bx; // in case sel is X or Z
    endcase
    zbar = ~z;
  end
endmodule
```

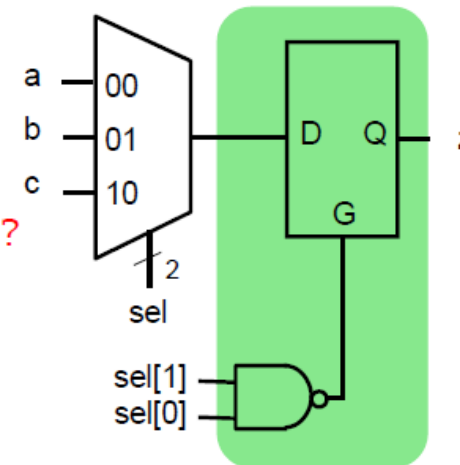
case looks for an exact bit-by-bit match of the value of the case expression (e.g., sel) against each case item, working through the items in the specified order. **casex/casez** statements treat X/Z values in the selectors as don't cares when doing the matching that determines which clause will be executed.

Unintentional Creation of State



Suppose there are multiple execution paths inside an `always` block, i.e., it contains `if` or `case` statements, and that on some paths a net is assigned and on others it isn't.

```
// 3-to-1 multiplexer ????  
module mux3(input a,b,c, input [1:0] sel, output reg z);  
  always @(*) begin  
    case (sel)  
      2'b00: z = a;  
      2'b01: z = b;  
      2'b10: z = c;  
      // if sel is 2'b11, no assignment to z!??  
    endcase  
  end  
endmodule
```



So sometimes `z` changes and sometimes it doesn't (and hence keeps its old value). That means the synthesized hardware has to have a way of remembering the state of `z` (i.e., its old value) since it's no longer just a combinational function of `sel`, `a`, `b`, and `c`. Not what was intended here. More on this in next lecture.

Keeping Logic Combinational



To avoid the unintentional creation of state, ensure that each variable that's assigned in an `always` block always gets assigned a new value at least once on every possible execution path.

```
// 3-to-1 multiplexer
module mux3(input a,b,c, input [1:0] sel, output reg z);
  always @ (*) begin
    z = 1'bx;    // a second assignment may happen below
    case (sel)
      2'b00: z = a;
      2'b01: z = b;
      2'b10: z = c;
      default: z = 1'bx;
    endcase
  end
endmodule
```

Use one or the other

It's good practice when writing combinational `always` blocks to provide a `default:` clause for each `case` statement and an `else` clause for each `if` statement.



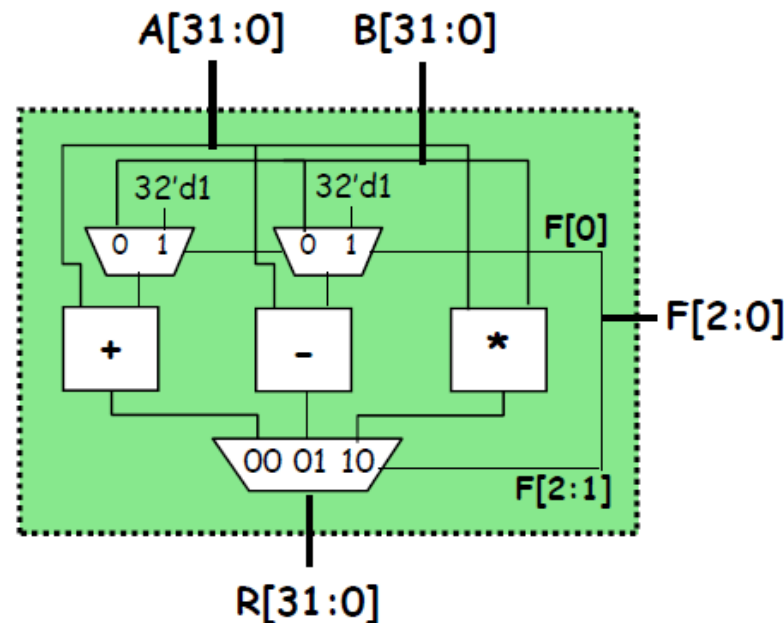
- Using Schematic
- Using gate if you know/have designed schematic
- Using “assign” if you have the Boolean equation or you know the logic straightforward
- Using “always” block
 - Using “if-else”
 - Using “case”

Defining Processor ALU in 5 Mins



- Modularity is essential to the success of large designs
- High-level primitives enable direct synthesis of behavioral descriptions (functions such as additions, subtractions, shifts (<< and >>), etc.

Example: A 32-bit ALU



Function Table

F2	F1	F0	Function
0	0	0	$A + B$
0	0	1	$A + 1$
0	1	0	$A - B$
0	1	1	$A - 1$
1	0	X	$A * B$

2-to-1 MUX

```
module mux32two
    (input [31:0] i0,i1,
     input sel,
     output [31:0] out);

    assign out = sel ? i1 : i0;
endmodule
```

32-bit Adder

```
module add32
    (input [31:0] i0,i1,
     output [31:0] sum);

    assign sum = i0 + i1;
endmodule
```

32-bit Subtractor

```
module sub32
    (input [31:0] i0,i1,
     output [31:0] diff);

    assign diff = i0 - i1;
endmodule
```

3-to-1 MUX

```
module mux32three
    (input [31:0] i0,i1,i2,
     input [1:0] sel,
     output reg [31:0] out);

    always @ (i0 or i1 or i2 or sel)
    begin
        case (sel)
            2'b00: out = i0;
            2'b01: out = i1;
            2'b10: out = i2;
            default: out = 32'bx;
        endcase
    end
endmodule
```

16-bit Multiplier

```
module mul16
    (input [15:0] i0,i1,
     output [31:0] prod);

    // this is a magnitude multiplier
    // signed arithmetic later
    assign prod = i0 * i1;

endmodule
```

Top Level ALU Declaration



- Given submodules:

```
module mux32two(i0,i1,sel,out);  
module mux32three(i0,i1,i2,sel,out);  
module add32(i0,i1,sum);  
module sub32(i0,i1,diff);  
module mul16(i0,i1,prod);
```

- Declaration of the ALU Module:

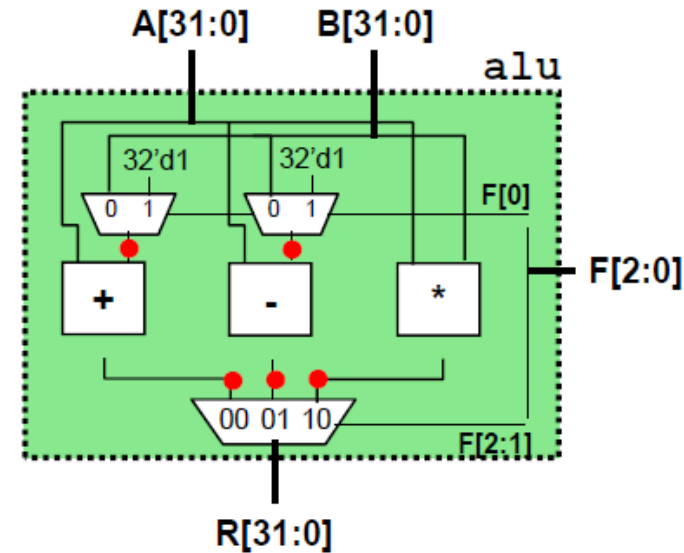
```
module alu  
  (input [31:0] a, b,  
   input [2:0] f,  
   output [31:0] r);  
  wire [31:0] addmux_out, submux_out;  
  wire [31:0] add_out, sub_out, mul_out;  
  mux32two adder_mux(b, 32'd1, f[0], addmux_out);  
  mux32two sub_mux(b, 32'd1, f[0], submux_out);  
  add32 our_adder(a, addmux_out, add_out);  
  sub32 our_subtractor(a, submux_out, sub_out);  
  mul16 our_multiplier(a[15:0], b[15:0], mul_out);  
  mux32three output_mux(add_out, sub_out, mul_out, f[2:1], r);  
endmodule
```

endmodule

module
names

(unique)
instance
names

corresponding
wires/regs in
module alu



intermediate output nodes

Modelsim / Testbench Introduction



Full Adder (1-bit)

```
module full_adder
(input a, b, cin,
 output reg sum, cout);

always @(a or b or cin)
begin
    sum = a ^ b ^ cin;
    cout = (a & b) | (a & cin) | (b & cin);
end
endmodule
```

Full Adder (4-bit)

```
module full_adder_4bit
( input[3:0] a, b,
  input cin,
  output [3:0] sum,
  output cout,
  wire c1, c2, c3;

    // instantiate 1-bit adders
    full_adder FA0(a[0],b[0], cin, sum[0], c1);
    full_adder FA1(a[1],b[1], c1, sum[1], c2);
    full_adder FA2(a[2],b[2], c2, sum[2], c3);
    full_adder FA3(a[3],b[3], c3, sum[3], cout);
endmodule
```

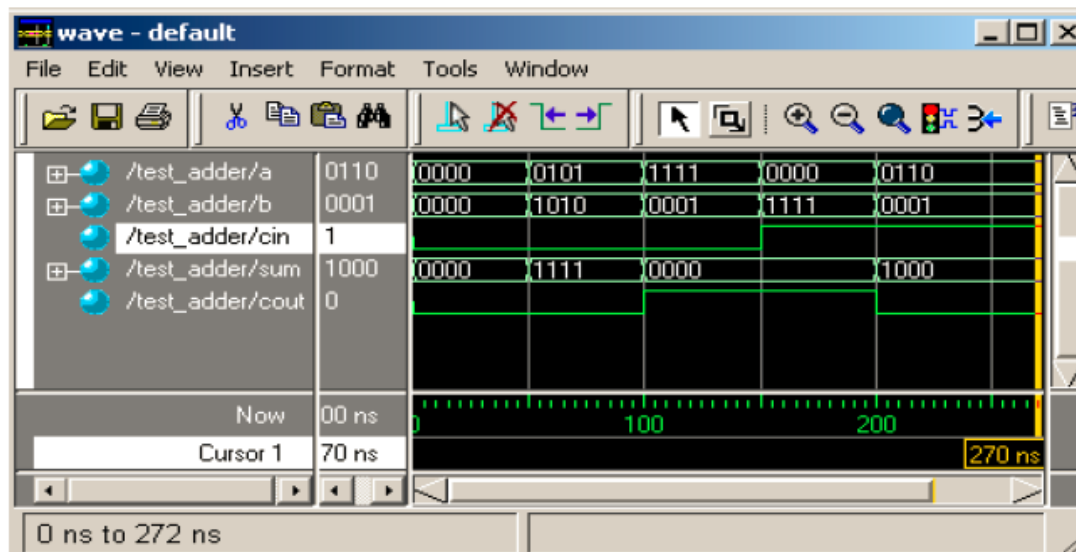
Testbench

```
module test_adder;
    reg [3:0] a, b;
    reg      cin;
    wire [3:0] sum;
    wire      cout;

    full_adder_4bit dut(a, b, cin,
                        sum, cout);

    initial
    begin
        a = 4'b0000;
        b = 4'b0000;
        cin = 1'b0;
        #50;
        a = 4'b0101;
        b = 4'b1010;
        // sum = 1111, cout = 0
        #50;
        a = 4'b1111;
        b = 4'b0001;
        // sum = 0000, cout = 1
        #50;
        a = 4'b0000;
        b = 4'b1111;
        cin = 1'b1;
        // sum = 0000, cout = 1
        #50;
        a = 4'b0110;
        b = 4'b0001;
        // sum = 1000, cout = 0
    end // initial begin
endmodule // test_adder
```

ModelSim Simulation



Before you get your hands on Xilinx Vivado...

See Verilog examples at

<https://vlsiverify.com/verilog/verilog-codes>

And go trial and error at

<https://edaplayground.com/>